UDA 7 · FUNZIONI E ARRAY IN C E C++

Il passaggio dei parametri

Una funzione riceve una copia dei tuoi dati o gli originali? Oggi scopriamo indirizzi e puntatori, senza paura.

PER COMINCIARE

Un amico deve sistemare la tua presentazione. Che cosa gli mandi?

SONDAGGIO

- **A** Una copia del file come allegato
- **B** Il link al file originale su Drive
- **C** Dipende da chi deve vedere le modifiche

CONDIVISIONE DI MEMORIA

Variabili locali e globali

```c
int punti = 0;       // globale

void bonus() {
    punti = punti + 10;
}
int main() {
    bonus();
    bonus();
    printf("Punti: %d\n", punti);
    return 0;
}
```

Locale

Dichiarata dentro una funzione: nasce e muore lì, le altre non la vedono.

Globale

Dichiarata fuori da tutte le funzioni: la **condividono tutte**. Comoda, ma chiunque può cambiarla.

QUIZ A TRABOCCHETTO

Che cosa stampa il main?

```c
int x = 5;              // globale

void cambia() {
    int x = 99;         // locale
}
int main() {
    cambia();
    printf("%d", x);
    return 0;
}
```

QUIZ

A 99

B 0

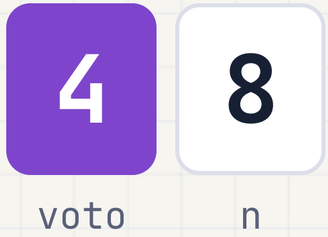
C Errore di compilazione

D 5

IL PASSAGGIO PER VALORE

La funzione riceve una copia

```c
void raddoppia(int n) {
    n = n * 2;
    printf("dentro: %d\n", n);
}
int main() {
    int voto = 4;
    raddoppia(voto);
    printf("fuori: %d\n", voto);
    return 0;
}
```



Stampa **dentro: 8** e poi **fuori: 4**.

> Il valore di `voto` viene **copiato** in `n`. La funzione lavora sulla copia: l'originale non cambia.

QUIZ

Lo scambio funziona?

```c
void scambia(int a, int b) {
    int t = a;
    a = b;
    b = t;
}
int main() {
    int x = 1, y = 2;
    scambia(x, y);
    printf("%d %d", x, y);
    return 0;
}
```

QUIZ

A 2 1

B 1 2

C 2 2

D 1 1

AL COMPUTER · 15 minuti

Caccia alla copia

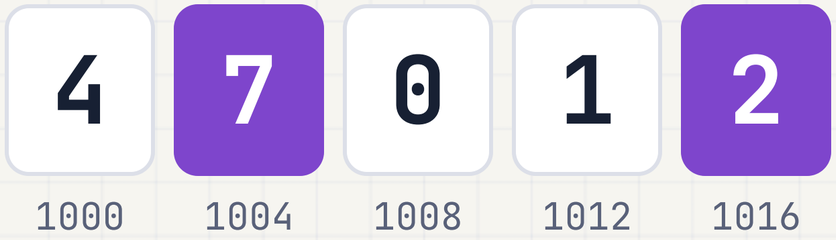
1. Su OnlineGDB (linguaggio C) scrivi il programma `raddoppia` della slide 5 ed eseguilo.
2. Aggiungi in `main` e in `raddoppia` una `printf` che stampa il valore **prima** e **dopo** il calcolo.
3. Ora trasformala in funzione: `int raddoppia(int n)` con `return n * 2;`
4. Nel `main` scrivi `voto = raddoppia(voto);` e stampa di nuovo.
5. Salva come **valore.c**.

Controllo: con la prima versione il main stampa `fuori: 4`; con la seconda stampa `8`. Il valore torna al main solo grazie al `return`.

IL TIPO PUNTATORE

Ogni variabile abita a un indirizzo

La memoria è come una lunga via: ogni cella ha un **numero civico**, l'indirizzo, e dentro c'è un valore.



`&a`

"Indirizzo di a": dove abita la variabile, per esempio 1004.

`int *p`

Un **puntatore**: una variabile che contiene un indirizzo.

IL TIPO PUNTATORE

Due simboli da ricordare: & e *

```c
int a = 7;
int *p = &a;   // p punta ad a

printf("%d", *p); // stampa 7

*p = 3;           // cambia a!
printf("%d", a);  // stampa 3
```

&a dà l'indirizzo della variabile a.

***p** è il valore che si trova all'indirizzo contenuto in p.

Perché ***p** e **a** sono la stessa cella di memoria.

PUZZLE

Abbina il codice al suo significato

`int *p;`	---	Il valore che si trova all'indirizzo contenuto in p
`&a`	---	Da ora p punta alla variabile a
`*p`	---	p non punta a niente
`p = &a;`	---	p è un puntatore a un intero
`p = NULL;`	---	L'indirizzo della variabile a

MENTI CHE CI HANNO PRECEDUTO

L'errore da un miliardo di dollari

> **Lo chiamo il mio errore da un miliardo di dollari: l'invenzione del riferimento nullo, nel 1965.**

Tony Hoare, informatico britannico, inventore del Quicksort, in una conferenza a Londra nel 2009



IL PASSAGGIO PER INDIRIZZO IN C

Ora lo scambio funziona

```c
void scambia(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}
int main() {
    int x = 1, y = 2;
    scambia(&x, &y);
    printf("%d %d", x, y);  // 2 1
    return 0;
}
```

La funzione riceve gli **indirizzi** di x e y, quindi lavora sugli **originali**.

> Ecco perché scrivi `scanf("%d", &x)`: scanf deve sapere **dove** mettere il numero letto!

PUZZLE

Metti in ordine: che cosa succede con scambia(&x, &y)?

1. `*b = t` scrive in y il vecchio valore di x
2. Il main passa a scambia gli indirizzi di x e y
3. `t = *a` copia in t il valore di x, cioè 1
4. Tornato nel main, x vale 2 e y vale 1
5. `*a = *b` scrive in x il valore di y, cioè 2
6. `a` riceve l'indirizzo di x, `b` quello di y

AL COMPUTER · 20 minuti

Metti in ordine due numeri

1. In C scrivi la funzione `void scambia(int *a, int *b)` della slide 12.
2. Scrivi `void ordina2(int *a, int *b)`: se `*a > *b` chiama `scambia(a, b)`.
3. Nel `main` leggi due numeri con `scanf`, chiama `ordina2(&x, &y)` e stampali.
4. Prova anche con due numeri uguali e con due negativi.
5. Salva come **indirizzo.c**.

Controllo: con 9 4 stampa `4 9`; con 3 8 stampa `3 8`; con −1 −5 stampa `-5 -1`.

IL PASSAGGIO PER INDIRIZZO IN C++

In C++ basta una & nel parametro

```
void scambia(int &a, int &b) {
    int t = a;
    a = b;
    b = t;
}
// nel main:
scambia(x, y);   // niente & qui
```

Il parametro `int &a` è un **riferimento**: un altro nome per la stessa variabile. Niente asterischi.

AL COMPUTER · 20 minuti

Il salvadanaio, in C++

1. Su OnlineGDB scegli **C++**.
2. Scrivi `void deposita(float &saldo, float importo)` che aggiunge l'importo al saldo.
3. Scrivi `void preleva(float &saldo, float importo)`: toglie l'importo solo se basta il saldo, altrimenti stampa `Saldo insufficiente`.
4. Nel `main`: saldo 0, deposita 20, preleva 5, preleva 50, poi stampa il saldo.
5. Salva come **salvadanaio.cpp**.

Controllo: compare una volta `Saldo insufficiente` e alla fine `Saldo: 15`. Prova a togliere la `&` da deposita: che cosa succede al saldo?

SFIDA FINALE

Apri il lucchetto con i parametri



- 1ª cifra: dopo lo scambio **per valore** della slide 6, quanto vale x
- 2ª cifra: **int a = 7; int *p = &a; *p = *p + 1;** quanto vale a
- 3ª cifra: quante & scrivi nella chiamata **scambia(&x, &y)**
- 4ª cifra: il valore di voto dopo **raddoppia(voto)** per valore, se voto era 4

codice

BIGLIETTO D'USCITA

Completa prima di uscire

Nel passaggio per valore la funzione riceve una _____ del dato.

L'operatore & restituisce l'_____ di una variabile.

Una variabile che contiene un indirizzo si chiama _____.

Una variabile dichiarata fuori da tutte le funzioni si chiama _____.

In C++ il parametro `int &a` si chiama _____.

⏱ 3:00

RIEPILOGO

Cosa portiamo a casa

- Locali dentro la funzione, globali condivise da tutte
- Per valore: la funzione lavora su una copia
- `&a` è l'indirizzo, `*p` il contenuto puntato
- Per indirizzo in C: `int *a` e chiamata con `&x`
- In C++: riferimento `int &a`
- La & di scanf ora ha un senso

Hai una domanda su una slide? Usa il pulsante **? Domande** in basso: la vedrà il docente e tutta la classe.